Philosophy of GSP_API Renderers

1. Preamble

GSP_API ships three renderer packages — `gsp_matplotlib`, `gsp_datoviz`, and `gsp_network` — and a fourth is plausible at any time (`gsp_webgpu`, `gsp_three`, `gsp_plotly`, ...). Read one of them and you have read the structure of all three: same subdirectory layout, same registration shape, same `RendererBase` contract, same per-visual file fan-out, same isinstance dispatch, same `(renderer, events, animator)` triad. The only thing each backend brings of its own is what happens *inside* `_render_visual()` and how `show()` blocks the process.

This document names the conventions that make the three packages a coherent ecosystem rather than three independent implementations, then walks through what is unique to each, then closes with a checklist for writing the fourth.

Audience. Someone reading a renderer to learn it, or someone about to write a fourth backend.

How to use this document. Each section names a pattern, points to the canonical file, and quotes the load-bearing lines. To verify a claim, open the cited file. To run the same example under all three renderers:

```
GSP_RENDERER=matplotlib python examples/points_example.py
GSP_RENDERER=datoviz    python examples/points_example.py
GSP_RENDERER=network    python examples/points_example.py    # needs the gsp_network
server running
```

---

2. The Five Design Principles

The "why" before the "what".

2.1 One interface, three backends

The contract is a six-method abstract base class — `RendererBase` at src/gsp/types/renderer_base.py:15-74:

```python
class RendererBase(ABC):
    @abstractmethod
    def __init__(self, canvas: Canvas): ...
    @abstractmethod
    def render(self, viewports, visuals, model_matrices, cameras) -> bytes: ...
    @abstractmethod
    def show(self) -> None: ...
    @abstractmethod
    def close(self) -> None: ...
    @abstractmethod
    def clear(self) -> None: ...
    @abstractmethod
    def get_canvas(self) -> Canvas: ...
```

`MatplotlibRenderer`, `DatovizRenderer`, and `NetworkRenderer` are the only direct subclasses. Everything else — registries, animators, examples — talks to the abstract type.

Payoff. Adding a backend is a question of subclassing, not of touching the core. The rest of the library is free of any `if backend == "matplotlib"` branch.

2.2 The three-fold triad

A renderer never ships alone. It always travels with a paired `ViewportEventsBase` and `AnimatorBase`, and the registry stores them together as a `RendererRegistryItem` (src/gsp/utils/renderer_registery.py:14-21):

```
@dataclass(frozen=True)
class RendererRegistryItem:
    renderer_name: str
    renderer_base_type: Type[RendererBase]
    viewport_event_base_type: Type[ViewportEventsBase]
    animator_base_type: Type[AnimatorBase]
```

The reason: events and animators are renderer-specific. Matplotlib's events come through `mpl_connect`; datoviz's come through its own callback registration; the network animator drives remote renders by reusing matplotlib's `FuncAnimation` locally. They cannot be ported across backends, so they must be co-versioned with the renderer they belong to.

Payoff. Once you have a renderer instance, the registry can hand back the matched events and animator without you naming the backend again — see `RendererRegistry._get_item_by_renderer_base`. One name, three classes, always in sync.

2.3 Self-registration on import

Each package's `__init__.py` calls `register_renderer_<name>()` at import time. There is no central manifest listing the available backends. src/gsp_matplotlib/**init**.py:15-17:

```
from .renderer_registration import register_renderer_matplotlib

register_renderer_matplotlib()
```

The registration function itself is six lines (src/gsp_matplotlib/renderer_registration.py:10-17):

```
def register_renderer_matplotlib():
    RendererRegistry.register_renderer(
        renderer_name="matplotlib",
        renderer_base_type=MatplotlibRenderer,
        viewport_event_base_type=ViewportEventsMatplotlib,
        animator_base_type=AnimatorMatplotlib,
    )
```

`gsp_datoviz` and `gsp_network` follow the same shape line-for-line.

Payoff. Installing a backend package is the only step needed to make it available. `pip install gsp_webgpu` — and the name `"webgpu"` exists in the registry the first time anything imports it.

2.4 Static-class dispatch, not a plugin registry

Per-visual rendering is a hand-written `isinstance` chain in `_render_visual()`, dispatching to a static class with a single `render(renderer, viewport, visual, model_matrix, camera)` method. Eight visual types today; eight `if/elif` arms (src/gsp_matplotlib/renderer/matplotlib_renderer.py:202-238):

```
def _render_visual(self, viewport, visual, model_matrix, camera):
    if isinstance(visual, Image):
        from gsp_matplotlib.renderer.matplotlib_renderer_image import RendererImage
        RendererImage.render(self, viewport, visual, model_matrix, camera)
    elif isinstance(visual, Pixels):
        from gsp_matplotlib.renderer.matplotlib_renderer_pixels import
RendererPixels
        RendererPixels.render(self, viewport, visual, model_matrix, camera)
    elif isinstance(visual, Points):
        ...
    else:
        raise NotImplementedError(...)
```

`DatovizRenderer._render_visual` (datoviz_renderer.py:185-219) has the same eight arms in the same order.

Payoff. The control flow is grep-able and traceable in five seconds. A registry would let third parties add visual types at runtime — a cost we don't need to pay for a fixed set of eight built-in visuals. To add a ninth visual to the project, you add one arm to each backend's `_render_visual()`.

2.5 Bytes out, not pixels out

`render()` returns `bytes` — PNG by default. Matplotlib accepts an `image_format` argument and plumbs it straight to `figure.savefig(format=...)`, so the same scene comes back as PNG, SVG, or PDF (matplotlib_renderer.py:107-200). The network renderer returns the PNG bytes it received over HTTP (network_renderer.py:96-167).

Payoff. The unified `bytes` return type is what makes `NetworkRenderer` possible without changing any caller. Whatever a local renderer would have produced, the network renderer can produce too — by paying postage instead of compute.

---

3. The Canonical Package Skeleton

Every renderer package has the same shape on disk. Use `src/gsp_matplotlib/` as the reference:

```
gsp_<backend>/
├── __init__.py                          # imports submodules, calls register_*
├── renderer_registration.py             # one function: register_renderer_<name>()
├── renderer/
│   ├── <backend>_renderer.py            # the RendererBase implementation
│   └── <backend>_renderer_<visual>.py   # one file per visual type (8 today)
├── animator/
│   └── animator_<backend>.py            # AnimatorBase implementation
├── viewport_events/
│   └── viewport_events_<backend>.py     # ViewportEventsBase implementation
└── utils/                               # backend-specific helpers (optional)
```

The mapping across the three backends is direct:

Concern	Shared base	gsp_matplotlib	gsp_datoviz	gsp
Renderer	`RendererBase`	`MatplotlibRenderer`	`DatovizRenderer`	`NetworkRe`
Animator	`AnimatorBase`	`AnimatorMatplotlib`	`AnimatorDatoviz`	`AnimatorN`
Events	`ViewportEventsBase`	`ViewportEventsMatplotlib`	`ViewportEventsDatoviz`	`ViewportE`
Registration	n/a	`renderer_registration.py`	`renderer_registration.py`	`renderer_`
Per-visual files	n/a	8 files, `matplotlib_renderer_*.py`	8 files, `datoviz_renderer_*.py`	none (zer dispatch)

Two notes on the table:

- **Network has no per-visual files.** It's a thin client — the whole scene is serialised and shipped over HTTP, so there is nothing to dispatch on. The eight per-visual renderers exist on the *server's* renderer (matplotlib or datoviz), not the client's.
- **`utils/` is backend-specific.** `gsp_matplotlib/utils/` holds `ConverterUtils` (GSP ↔ matplotlib types) and `RendererUtils` (face culling, normals); `gsp_datoviz/utils/` holds its own converter; `gsp_network/tools/` holds the Flask server (`network_server.py`) and a port-kill utility. These directories carry no contract — only the renderer/animator/events triad does.

---

4. The `RendererBase` Contract — Method by Method

Six abstract methods. Read `MatplotlibRenderer` as the reference implementation; the other two are minor variations.

`__init__(canvas: Canvas)`

Store the canvas, allocate per-renderer state. Matplotlib creates a `Figure` sized from the canvas dimensions and DPI (matplotlib_renderer.py:46-68); datoviz creates a `dvz.App` and a `dvz_figure` (datoviz_renderer.py:47-69); network creates a local matplotlib figure to *display* the bytes it will receive (network_renderer.py:44-70) and additionally takes `server_base_url` and `remote_renderer_name` arguments — the only constructor in the family that takes more than `canvas` .

`render(viewports, visuals, model_matrices, cameras) -> bytes`

The workhorse. The four sequences are parallel — they must all have the same length, and matplotlib asserts it explicitly (matplotlib_renderer.py:136-138):

```
assert (
    len(viewports) == len(visuals) == len(model_matrices) == len(cameras)
), f"All length MUST be equal. Mismatched lengths: ..."
```

The rendering loop is then a single `zip` (matplotlib_renderer.py:182-183):

```
for viewport, visual, model_matrix, camera in zip(viewports, visuals,
model_matrices, cameras):
```

```
    self._render_visual(viewport, visual, model_matrix, camera)
```

Datoviz repeats this exact shape at datoviz_renderer.py:147-148. Network skips per-visual dispatch entirely and instead serialises the whole scene at network_renderer.py:120-126.

Two backends extend the signature: matplotlib adds `return_image: bool = True, image_format: str = "png"` ; datoviz adds the same pair (but accepts only `"png"`). Network does not — it always returns the PNG it received.

`show() -> None`

Blocking. Matplotlib calls `matplotlib.pyplot.show()` (matplotlib_renderer.py:90-101); datoviz registers a 'q'-to-quit keyboard handler and calls `self._dvz_app.run()` (datoviz_renderer.py:87-102); network reuses matplotlib's `pyplot.show()` to display the bytes it received (network_renderer.py:169-176). All three short-circuit when the `GSP_TEST=True` environment variable is set, so test runs never hang.

`close() -> None`

Release resources. Matplotlib stops the event loop and closes the figure (matplotlib_renderer.py:78-88); datoviz calls `self._dvz_app.destroy()` (datoviz_renderer.py:71-73); network mirrors matplotlib because that's where its display lives (network_renderer.py:80-86).

`clear() -> None`

Wipe the current frame. Matplotlib calls `figure.clf()` ; network does the same. Datoviz currently raises `NotImplementedError` (datoviz_renderer.py:104-109) — the GSP scene doesn't currently need clearing because per-visual renderers update existing artists in place rather than rebuild.

`get_canvas() -> Canvas`

Return the `Canvas` passed to `__init__` . One line in every backend.

---

5. The Per-Visual Renderer Pattern

This is the load-bearing convention. The main `<backend>_renderer.py` orchestrates; the eight `<backend>_renderer_<visual>.py` files do the actual drawing.

5.1 The dispatch table is `isinstance` , not a dict

`_render_visual()` is the same shape in every backend that does local rendering. Matplotlib at matplotlib_renderer.py:202-238 and datoviz at datoviz_renderer.py:185-219 both use the same eight if/elif arms in the same order: `Image, Pixels, Points, Paths, Markers, Mesh, Segments, Texts` . The per-visual modules are imported lazily inside each arm — first to avoid eager imports of optional dependencies, second because the import itself is the dispatch table. To add a ninth visual, you add one arm to each `_render_visual()` and create one new `<backend>_renderer_<newvisual>.py` per backend.

5.2 The per-visual class is static and stateless

Each `<backend>_renderer_<visual>.py` exports one class with a single static method:

```
class RendererPoints:
    @staticmethod
```

```
    def render(renderer, viewport, points, model_matrix, camera):
        ...
```

State lives on the *main* renderer, not on the per-visual class. Matplotlib stores artists in `self._artists` keyed by `f"{viewport_uuid}_{visual_uuid}"` (matplotlib_renderer.py:54-58); datoviz does the same in `self._dvz_visuals` (datoviz_renderer.py:61-69). The per-visual file is a *code-organisation unit*, not a lifecycle owner.

5.3 Lazy create, mutate-update

The first `render()` call for a given `(viewport, visual)` pair creates the underlying artist or GPU object; subsequent calls mutate it in place. This is what makes the animator efficient — return only the changed visuals from your `@animator.event_listener` callback and the renderer does no rebuild work for the rest. The pattern hinges on the same `f"{viewport_uuid}_{visual_uuid}"` cache key in both backends, so a per-visual renderer can ask "have I seen this pair before?" with one dict lookup.

---

6. The Registry and Discovery

`RendererRegistry` is 100 lines (src/gsp/utils/renderer_registery.py:24-99). Three entry points the rest of the library uses.

`register_renderer(name, renderer_type, events_type, animator_type)` — called once per package at import time. Stores the triad in a dict keyed by name.

`create_renderer(name, canvas)` — what `ExampleHelper.create_renderer` is built on. The example helper itself (examples/common/example_helper.py:55-74) currently uses an `if/elif` chain rather than the registry — both work; the registry is the more general mechanism, the helper is the more readable one for example code.

`create_viewport_events(renderer_base, viewport)` and **`create_animator(renderer_base)`** — these look up the triad by *instance type*, not by name (renderer_registery.py:93-99):

```
@staticmethod
def _get_item_by_renderer_base(renderer_base):
    for item in RendererRegistry._registry.values():
        if isinstance(renderer_base, item.renderer_base_type):
            return item
    raise ValueError(...)
```

This is why the triad is enforced. Once you have a renderer, the registry can hand you the matching events and animator without you ever naming the backend a second time. If a package shipped a renderer without its paired events/animator, this lookup would fail.

---

7. Backend-Specific Notes

What is *unique* to each backend, kept short on purpose. The shared structure is in §3-6; everything below is the local colour.

7.1 gsp_matplotlib

Reference: matplotlib_renderer.py:107-200.

- **One `Axes` per viewport, lazily created and cached** in `self._axes` keyed by viewport UUID (matplotlib_renderer.py:143-176). Each axes is positioned in normalized figure coordinates, with `xlim/ylim` set to `(-1, 1)` to match GSP's NDC convention.
- **Multi-format output** via `image_format=` plumbed straight to `figure.savefig(format=image_format, dpi=...)` (line 194). PNG, SVG, PDF, JPG all work — the user picks at the `render()` call.
- **Animator wraps `matplotlib.animation.FuncAnimation`** and is the natural fit for video export (`FuncAnimation.save()` with ffmpeg or pillow writers chosen by file extension).
- **Events go through `mpl_connect`** (viewport_events_matplotlib.py) and are clipped per-viewport, since multiple viewports share one matplotlib canvas.

7.2 gsp_datoviz

Reference: datoviz_renderer.py:114-179.

- **GPU resources cached in `self._dvz_visuals`** keyed by `f"{viewport_uuid}_{visual_uuid}"` , with a `_dvz_panels` cache for the per-viewport datoviz `Panel` (datoviz_renderer.py:225-244). The "create once, mutate forever" rule from §5.3 matters most here because GPU resource churn would dominate frame cost.
- **MVP transform happens on the CPU** before vertices are handed to datoviz — the GPU side sees pre-transformed positions, not matrices. Carry that in mind if you intend to wire datoviz visuals into a custom shader pipeline.
- **Y-axis flip** between datoviz (top-left origin) and GSP (bottom-left). Visible in `_getOrCreateDvzPanel` at line 232: `dvz_offset = (viewport.get_x(), self.get_canvas().get_height() - viewport.get_y() - viewport.get_height())` .
- **Screenshots require offscreen mode.** If `render(..., return_image=True)` is called on an interactive `dvz.App` , the renderer transparently spawns a temporary offscreen `DatovizRenderer` , renders into it, captures the PNG, and destroys it (lines 168-177).

7.3 gsp_network

Reference: network_renderer.py:96-167 and network_server.py.

- **Pure thin client.** `render()` calls `PydanticSerializer.serialize(...)` to convert the whole scene into a JSON-friendly dict, POSTs it to `/render` , and decodes the PNG response into a local matplotlib figure for display. No per-visual dispatch on the client side — anything not serializable by `PydanticSerializer` cannot be sent.
- **Server side** (network_server.py:51-115) is a Flask app with a single `/render` endpoint. The payload's `renderer_name` field selects which local renderer to instantiate (lines 75-79) — `MatplotlibRenderer` or `DatovizRenderer(offscreen=True)` . The server depends on those packages directly; the client does not.
- **Asymmetric events.** Client input (mouse, keyboard) is captured locally via `ViewportEventsNetwork` and **does not propagate to the server**. Interactive controls (camera orbit, pan/zoom) execute locally; only the resulting frame request crosses the wire.
- **Full-scene transport per frame.** No delta encoding; every `render()` ships the entire serialized scene. Suitable for moderate-rate interactive sessions, not real-time streaming.

---

8. Writing a New Renderer: A Checklist

The "now you do it" section. Each item maps to a convention named above.

- [ ] **Pick a name and create the package.** `src/gsp_<name>/` with subdirectories `renderer/`, `animator/`, `viewport_events/`, plus `renderer_registration.py` and `__init__.py`. Mirror `src/gsp_matplotlib/` — copy the layout, rename the files.
- [ ] **Subclass `RendererBase`.** Implement the six abstract methods. `MatplotlibRenderer` is the most readable reference; `NetworkRenderer` is the smallest.
- [ ] **Decide your dispatch.** If you can lean on a host library that has primitives for points/lines/meshes (matplotlib, datoviz, plotly, three.js), follow the per-visual-file pattern from §5: one file per visual type, hand-written `isinstance` chain in `_render_visual()`, static `render(renderer, viewport, visual, model_matrix, camera)` method on each. If you're rendering from scratch (e.g. raw OpenGL), the per-visual files are still the right unit — they just hold shader setup instead of library calls. If you're a thin client like `gsp_network`, you can skip the per-visual files entirely and serialise the whole scene.
- [ ] **Cache lazily, mutate not rebuild.** Store per-visual artist or GPU handles in `self._artists` (or `self._<backend>_visuals`) keyed by `f"{viewport_uuid}_{visual_uuid}"`. First render creates; subsequent renders update. This is what lets the animator be efficient (§5.3).
- [ ] **Subclass `AnimatorBase` and `ViewportEventsBase`.** They must be co-versioned with the renderer; the registry pairs them by instance type (§6). `AnimatorNetwork` is the simplest reference because it just reuses matplotlib's `FuncAnimation` loop.
- [ ] **Self-register on import.** Write `renderer_registration.py:register_renderer_<name>()` calling `RendererRegistry.register_renderer(...)` with all three classes; have `__init__.py` invoke that function at module top level. Six lines each, copy from `gsp_matplotlib`.
- [ ] **Add the name to `ExampleHelper`.** Update the `Literal[...]` type alias and the `if/elif` arms in examples/common/example_helper.py — `get_renderer_name`, `create_renderer`, `create_animator`, `create_viewport_events`. Three places, all near each other.
- [ ] **Verify under an existing example.** `GSP_RENDERER=<your_name> python examples/points_example.py` should produce a PNG in `examples/output/` visually equivalent to the matplotlib output. Cross-checking against an existing backend is what turns the example suite into a conformance test (see philosophy_examples.md §2.1).

---

9. Verification: how to read this document against the code

Every claim above is grounded in a file. The shell commands below confirm the structural claims directly.

1. **Parallel package layout (§3)** — same subdirectories, same files in each backend:

```
ls src/gsp_matplotlib src/gsp_datoviz src/gsp_network
```

2. **Same eight per-visual files in matplotlib and datoviz (§5.1)** — datoviz and matplotlib mirror each other:

```
ls src/gsp_matplotlib/renderer/ src/gsp_datoviz/renderer/
```

3. **Three registration sites, same shape (§2.3)**:

```
grep -n "RendererRegistry.register_renderer"
src/gsp_*/renderer_registration.py
```

4. **Same isinstance dispatch chain in both local backends (§5.1)**:

```
grep -n "isinstance(visual,"
src/gsp_matplotlib/renderer/matplotlib_renderer.py
src/gsp_datoviz/renderer/datoviz_renderer.py
```

5. `RendererBase` **is the only inheritance edge (§2.1)**:

```
grep -rn "class.*RendererBase" src/gsp_*/renderer/
```

6. **The triad is enforced by the registry (§2.2)** — read `_get_item_by_renderer_base` in:

```
sed -n '93,99p' src/gsp/utils/renderer_registery.py
```

7. `GSP_RENDERER` **is the only env-var gate (§1)** — example code reads it:

```
grep -n "GSP_RENDERER" examples/common/example_helper.py
```

When the code drifts from this document — change the document. The renderer packages are the source of truth; this file just names what they already do.